CORE JAVA SERIES — DAY 5

Arrays, Strings & Memory Internals

Why arrays exist, how to declare and use 1D/2D/3D arrays and Strings, then a deep dive into how the JVM actually stores arrays in memory and gives us instant random access.

Why Arrays Exist

Storing many related values (e.g. roll numbers of 100 students) by declaring one variable per value doesn't scale. An **array** solves this: a single block of **contiguous memory** (one location right after another) is reserved upfront, divided into equal-sized sections — one per value.

> **Array, plainly:** a collection of a particular data type. Every element in the collection must be the same type.

Declaring and Defining an Array



`int[] rollNumbers = new int[3];` — data type, square brackets (signals "array"), name, then new + type + size.

- **Data type** — what each element holds (here, int).
- **Square brackets** — tell the compiler this is an array, not a plain variable.
- **Identifier** — the array's name.
- `new` — a special keyword that allocates memory in the **heap** (covered in depth once objects/classes are studied).
- **Size** — how many elements to reserve room for; the compiler needs to know this upfront so it can allocate the right amount of contiguous memory.

> **Two valid declaration styles:** `int[] arr` or `int arr[]` — the square brackets can attach to the data type or the variable name. Java prefers the former; the latter exists for legacy compatibility with C/C++ syntax.

Indexing Starts at Zero

Each cell in the array gets an **index** so it can be individually addressed — the first cell is index `0`, not `1`. To store: `rollNumbers[0] = 1001;`. To retrieve/print: `System.out.println(rollNumbers[0]);`.

Filling and Printing with Loops

Manually writing one line per index doesn't scale any better than one variable per value did. Loops solve this — `for (int i = 0; i < 3; i++) { rollNumbers[i] = x; x++; }` fills every slot in one compact block, and the same loop pattern prints every element back out.

.length — Never Hard-Code the Size

> `arrayName.length` returns an array's size dynamically — e.g. `rollNumbers.length` gives 3. Using this in a loop's condition (`i < rollNumbers.length`) instead of hard-coding a number means the loop keeps working correctly even if the array's size changes later.

ArrayIndexOutOfBoundsException

> **Accessing an invalid index** (negative, or ≥ the array's length) throws this exception — Java's way of signaling something went wrong, rather than silently reading random memory.

Active Recall — Part 1

Q1. Why does an array need contiguous memory?

A. So each element's location can be calculated directly from a starting address, instead of scattering elements randomly across memory.

Q2. What is the new keyword used for when declaring an array?

A. It allocates the array's memory in the heap — arrays are a non-primitive data type.

Q3. Why should you use arrayName.length instead of hard-coding the size in a loop?

A. Because the array's size might change later, or you might not know it in a large codebase — .length always reflects the current size.

Q4. What exception occurs when accessing an index outside an array's valid range?

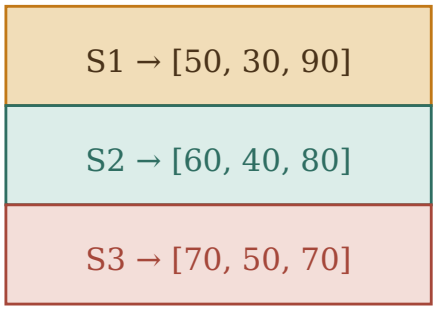
A. ArrayIndexOutOfBoundsException.

Multidimensional Arrays

A standard array (1D) is a collection where every element is a single value of some type. A **2D array** is an **"array of arrays"** — every element of the outer array is itself a whole array.

Why 2D Arrays Are Needed

Example: storing marks for 3 students, each with marks in 3 different subjects. A single 1D array can't hold "3 marks per student" in one cell — but if each cell itself holds a small array of 3 marks, the whole dataset fits naturally.



Conceptually: 3 rows (students), each row is itself an array of 3 columns (subject marks) — a 3×3 matrix.

Declaring a 2D Array

`int[][] marks = new int[3][3];` — two square brackets signal a 2D array. The first number is **rows**, the second is **columns**. Access with two indices: `marks[row][col]` — e.g. `marks[0][0] = 50;`.

Printing with Nested Loops

An outer loop walks the rows; an inner loop walks the columns within each row: `for (int row = 0; row < marks.length; row++) { for (int col = 0; col < marks[row].length; col++) { print marks[row][col]; } println(); }`. Note: `marks[row].length` gives that specific row's own length — important once rows can have different sizes.

Jagged Arrays — Rows of Different Lengths

Since each row is genuinely its own separate array, rows don't have to be the same length. Declare rows individually: `marks[0] = new int[1]; marks[1] = new int[2]; marks[2] = new int[3];` — a 2D array where row 0 has 1 column, row 1 has 2, row 2 has 3. Useful when, e.g., different students take different numbers of subjects.

Row count is required; column count is optional at declaration time — you can declare just `new int[3][]` (3 rows, sizes to be defined later per row) when rows will have different lengths.

3D Arrays and Beyond

A **3D array** is an "array of arrays of arrays": `int[][][] arr = new int[3][3][3];`. Conceptually, imagine stacking multiple 2D matrices (layers) on top of each other — like stacking flat squares to build a cube. Java supports any number of dimensions, though 3D is already uncommon in practice and beyond that is rare.

Array Literal Initialization

When values are known upfront, skip `new` entirely:

> `int[] rollNumbers = {4, 5, 6};` (1D) or `int[][] marks = {{4,5,6}, {7,8,9}, {23,34,93}};` (2D) — the compiler infers the size from how many values are listed.

Strings (Introduction)

`String` is a non-primitive data type — a proper deep dive comes later (after Object-Oriented Programming), but the basics: declare with `String name = "text";` — string literals always go in **double quotes** (unlike `char`, which uses single quotes).

> **The `+` operator on Strings means concatenation**, not addition — `firstName + lastName` joins the two strings together with no automatic spacing; add a literal `" "` in between if a space is wanted: `firstName + " " + lastName`.

Active Recall — Part 2

Q1. What does it mean to say a 2D array is an "array of arrays"?

A. The outer array's elements are not plain values — each element is itself a complete, separate array.

Q2. In int[][] marks = new int[3][4], what do the 3 and 4 represent?

A. 3 is the number of rows; 4 is the number of columns per row.

Q3. How do you create a 2D array where each row has a different length?

A. Declare only the row count upfront (new int[3][]), then assign each row its own separately-sized array individually (marks[0] = new int[2], etc.).

Q4. What does + do when used between two String values?

A. Concatenates them (joins them together) — it does not perform addition.

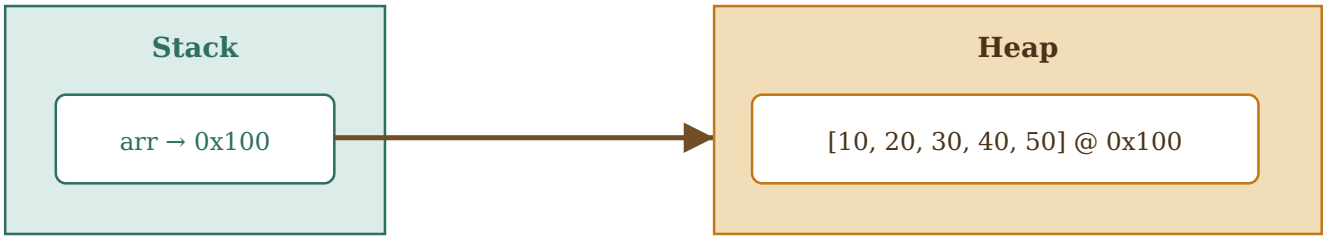
How Arrays Are Actually Stored: Random Access

Accessing `arr[2]` doesn't require the JVM to loop through index 0, then 1, to reach index 2 — it jumps **directly** there. This capability is called **random access**. Understanding how requires first understanding where different data types live in memory.

Primitives vs. Non-Primitives in Memory

Primitives	Non-Primitives
int, float, boolean, char, etc. — stored directly in Stack memory	Arrays, Strings, objects — actual data lives in Heap memory

When you write `int x = 4;`, `x` directly holds the value 4, in stack memory. But when you write `int[] arr = new int[5];`, the array itself is created in the **heap**, and `arr` (in the stack) is a **reference variable** — it doesn't hold the array directly, it just stores the array's starting memory address (points to it, like an arrow).



A reference variable in the stack points to the array's actual data sitting in the heap.

The Random Access Formula

Since memory is contiguous and every element of a given type takes a fixed number of bytes, any element's exact address can be calculated directly:

```
address of arr[i] = base address + (size of data type × i)
```

Example: int (4 bytes) array starting at address 100 — `arr[3]` → 100 + (4 × 3) = 112. The JVM jumps straight to byte 112 and reads 4 bytes — no traversal needed. This is exactly why arrays give O(1) random access.

Data Type Sizes Used in the Formula

Type	Size Used
byte	1 byte
short, char	2 bytes
int, float	4 bytes
long, double	8 bytes
reference (array/object element)	4 bytes

Boolean: The Exception

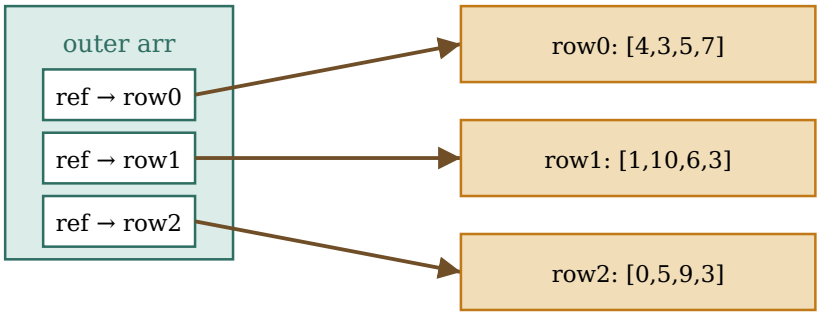
boolean has no officially fixed size in the Java specification — it's left to each JVM implementation to decide, based on what's most efficient for that platform's CPU to fetch. Most common JVMs (HotSpot, OpenJDK) use **1 byte** per boolean, since a single bit could technically represent true/false, but CPUs fetch memory in byte-sized chunks (not individual bits), so rounding up to 1 byte is more efficient than any bit-level packing scheme.

Bounds Checking Prevents Random Memory Access

Before applying the formula, Java checks: `if (index < 0 || index >= array.length) throw ArrayIndexOutOfBoundsException;`. This stops the formula from ever computing an address **outside** the array's allocated block — which could otherwise land on memory being used for something else entirely.

2D Arrays in Memory: Arrays of References

Java has no built-in concept of "rows and columns" — that's purely how *humans* visualize the data (as a matrix/table), the way we also do with math or SQL tables. Internally, a 2D array is truly an **array of arrays**: the outer array's elements are not integers — they are **reference variables**, each pointing to a separate, independent array elsewhere in the heap.



The outer array holds references (4 bytes each); each reference points to its own independently-stored row array.

Fetching arr[1][2]: The Formula Applied Twice

- **Step 1 (find the row):** address = base(outer array) + (4 bytes × 1) → gives the reference stored at that slot.
- **Step 2 (follow the reference, find the column):** address = base(that row's array) + (4 bytes × 2) → gives the actual int value.

> **Key insight:** the outer array stores 4-byte *references*, not the actual row data — the row data lives at whatever separate heap address each reference points to. This is why jagged (different-length) rows are possible: each row is a fully independent array.

Strings in Memory & String Arrays

Since `String` is also non-primitive, a `String[]` works exactly like a 2D array conceptually: the array stores **references** (4 bytes each), each pointing to a separately-allocated String object elsewhere in the heap.

> **Historical note:** up through JDK 8, a String was internally implemented as a character array (`char[]`) — so `String` and "array of characters" were essentially equivalent under the hood. From JDK 9 onward, String's internal representation changed (byte-array-based, for memory efficiency) and is no longer exactly a plain char array — a detail covered in full once Strings are studied in depth.

Why Random Access Matters: Caching

CPUs fetch memory in chunks of several bytes at a time, not one byte at a a time. When you read `arr[3]` (say, 4 bytes), the CPU may fetch extra adjacent bytes along with it and store them in its own fast on-chip memory (**registers/cache**). If your next access is `arr[4]`, sitting right next to it in contiguous memory, that data may already be cached — avoiding a slower trip back to RAM entirely.

> **Bottom line:** contiguous memory doesn't just enable the O(1) address formula — it also plays well with CPU caching, since nearby elements are likely to be fetched together, making arrays one of the most hardware-friendly data structures.

Active Recall — Part 3

Q1. Why does an int variable directly hold its value, but an array variable doesn't directly hold the array?

A. int is a primitive type stored directly in the stack. An array is non-primitive — its actual data lives in the heap, and the stack variable is only a reference (a pointer to the heap address).

Q2. What is the formula for finding the memory address of arr[i]?

A. base address + (size of the data type × i).

Q3. Why doesn't Java specify a fixed byte size for boolean?

A. It's left to each JVM implementation to choose based on what's most CPU-efficient on that platform; most

common JVMs use 1 byte.

Q4. In a 2D array, what does the outer array actually store — the row data, or something else?

A. It stores reference variables (4 bytes each) — each pointing to a separately-stored row array elsewhere in the heap, not the row's actual values.

Q5. How is fetching a value from a String array different from fetching from an int array?

A. An int array stores the actual int values directly. A String array stores references — each element points to a separately-allocated String object in the heap that must be followed to get the actual text.

Q6. Besides enabling direct address calculation, what other hardware benefit does contiguous memory provide?

A. CPU caching — nearby elements are likely fetched together into fast on-chip memory, so subsequent accesses to adjacent indices can avoid a slower round-trip to RAM.
